

React Coding Round — Machine Coding Cheatsheet

Commonly asked React interview coding questions with complete, copy-paste ready solutions.

Table of Contents

1. [Counter](#)
 2. [Todo App \(add / delete / toggle / edit\)](#)
 3. [Fetch API data + searchable users list](#)
 4. [Parent → Child props \(minimal version\)](#)
 5. [Custom Hooks](#)
 6. [Quick talking points](#)
-

1. Counter

```
import { useState } from "react";

function Counter({ step = 1 }) {
  const [value, setValue] = useState(0);
  const bump = (delta) => setValue((prev) => prev + delta);

  return (
    <section>
      <p>Current value: {value}</p>
      <button onClick={() => bump(step)}>Increase</button>
      <button onClick={() => bump(-step)} disabled={value === 0}>
        Decrease
      </button>
      <button onClick={() => setValue(0)}>Clear</button>
    </section>
  );
}

export default Counter;
```

2. Todo App (add / delete / toggle / edit)

Features: Add task, Delete task, Mark complete, Edit task **Must know:** `useState`, `map`, `filter`

```
import { useState } from "react";

let nextId = 1;

function TaskBoard() {
  const [tasks, setTasks] = useState([]);
```

```

const [draft, setDraft] = useState("");
const [editingId, setEditingId] = useState(null);

function saveTask() {
  const title = draft.trim();
  if (title === "") return;

  setTasks((prev) => {
    if (editingId === null) {
      return prev.concat({ id: nextId++, title, finished: false });
    }
    return prev.map((task) =>
      task.id === editingId ? { ...task, title } : task,
    );
  });

  setDraft("");
  setEditingId(null);
}

function updateTask(id, changes) {
  setTasks((prev) =>
    prev.map((t) => (t.id === id ? { ...t, ...changes } : t)),
  );
}

function dropTask(id) {
  setTasks((prev) => prev.filter((t) => t.id !== id));
  if (editingId === id) {
    setEditingId(null);
    setDraft("");
  }
}

function startEdit(task) {
  setEditingId(task.id);
  setDraft(task.title);
}

const remaining = tasks.reduce((n, t) => (t.finished ? n : n + 1), 0);

return (
  <div>
    <input
      value={draft}
      placeholder="What needs doing?"
      onChange={(e) => setDraft(e.target.value)}
      onKeyDown={(e) => e.key === "Enter" && saveTask()}
    />
    <button onClick={saveTask}>{editingId === null ? "Add" : "Save"}</button>

    {tasks.length === 0 && <p>No tasks yet.</p>}

    <ul>

```

```

    {tasks.map((task) => (
      <li key={task.id}>
        <label>
          <input
            type="checkbox"
            checked={task.finished}
            onChange={() =>
              updateTask(task.id, { finished: !task.finished })
            }
          />
          <span className={task.finished ? "done" : ""}>{task.title}</span>
        </label>
        <button onClick={() => startEdit(task)}>Edit</button>
        <button onClick={() => dropTask(task.id)}>Remove</button>
      </li>
    ))}
  </ul>

  <small>
    {remaining} remaining of {tasks.length}
  </small>
</div>
);
}

export default TaskBoard;

```

3. Fetch API data + searchable users list

```

import { useEffect, useState } from "react";

const API = "https://jsonplaceholder.typicode.com/users";

function UserDirectory() {
  const [people, setPeople] = useState([]);
  const [term, setTerm] = useState("");
  const [status, setStatus] = useState("loading"); // loading | ready | failed

  useEffect(() => {
    const controller = new AbortController();

    async function load() {
      try {
        const response = await fetch(API, { signal: controller.signal });
        if (!response.ok) throw new Error("Request failed: " + response.status);
        const data = await response.json();
        setPeople(data);
        setStatus("ready");
      } catch (err) {
        if (err.name !== "AbortError") setStatus("failed");
      }
    }
  }, [API]);
}

```

```

    }
  }

  load();
  return () => controller.abort();
}, []);

const needle = term.trim().toLowerCase();
const visible = needle
  ? people.filter((p) =>
    [p.name, p.email, p.company?.name]
      .join(" ")
      .toLowerCase()
      .includes(needle),
    )
  : people;

if (status === "loading") return <p>Loading users...</p>;
if (status === "failed") return <p>Could not load users.</p>;

return (
  <div>
    <input
      type="search"
      value={term}
      placeholder="Search by name, email or company"
      onChange={(e) => setTerm(e.target.value)}
    />

    <p>{visible.length} result(s)</p>

    {visible.map((person) => (
      <UserCard key={person.id} user={person} />
    ))}

    {visible.length === 0 && <p>Nothing matched "{term}".</p>}}
  </div>
);
}

// Parent -> Child props example
function UserCard({ user }) {
  return (
    <article>
      <h4>{user.name}</h4>
      <p>{user.email}</p>
      <p>{user.company?.name}</p>
    </article>
  );
}

export default UserDirectory;

```

4. Parent → Child props (minimal version)

```
function Greeting({ name, role, onPing }) {
  return (
    <div>
      <p>
        {name} - {role}
      </p>
      <button onClick={() => onPing(name)}>Ping</button>
    </div>
  );
}

export default function Parent() {
  const team = [
    { id: 1, name: "Asha", role: "Dev" },
    { id: 2, name: "Ravi", role: "QA" },
  ];

  const handlePing = (who) => alert("Pinged " + who);

  return (
    <div>
      {team.map((m) => (
        <Greeting key={m.id} name={m.name} role={m.role} onPing={handlePing} />
      ))}
    </div>
  );
}
```

5. Custom Hooks

useFetch

```
import { useEffect, useState } from "react";

export function useFetch(url) {
  const [state, setState] = useState({
    data: null,
    loading: true,
    error: null,
  });

  useEffect(() => {
    let alive = true;
    setState({ data: null, loading: true, error: null });

    fetch(url)
```

```

    .then((r) => {
      if (!r.ok) throw new Error(r.statusText);
      return r.json();
    })
    .then((data) => alive && setState({ data, loading: false, error: null }))
    .catch(
      (error) => alive && setState({ data: null, loading: false, error }),
    );

    return () => {
      alive = false;
    };
  }, [url]);

  return state;
}

```

useDebounce (great for search inputs)

```

import { useEffect, useState } from "react";

export function useDebounce(value, delay = 400) {
  const [settled, setSettled] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setSettled(value), delay);
    return () => clearTimeout(timer);
  }, [value, delay]);

  return settled;
}

```

Using both together

```

function SearchableUsers() {
  const { data, loading, error } = useFetch(
    "https://jsonplaceholder.typicode.com/users",
  );
  const [term, setTerm] = useState("");
  const debounced = useDebounce(term, 300);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error.message}</p>;

  const list = data.filter((u) =>
    u.name.toLowerCase().includes(debounced.toLowerCase()),
  );

  return (

```

```
    <>
      <input value={term} onChange={(e) => setTerm(e.target.value)} />
      {list.map((u) => (
        <UserCard key={u.id} user={u} />
      ))}
    </>
  );
}
```

Quick talking points

- `useState` updater form (`setX(prev => ...)`) avoids stale state in batched updates.
- `key` must be a stable unique id, not the array index.
- `filter` returns a new array — never mutate state directly.
- Cleanup in `useEffect` (`AbortController` / flag) prevents setting state after unmount.
- Custom hooks = any function starting with `use` that calls other hooks; they share logic, not state.